

Guion de presentación — Krypton: Migración a Microservicios

Guion slide-por-slide para armar el PowerPoint. Cada slide trae: **lo que va en la diapositiva** (bullets cortos),  **qué decir** (notas del orador) y  **visual sugerido**. Está repartido entre los **5 integrantes** según quién desarrolló cada parte.

Duración objetivo: ~20–25 min (≈1 min/slide) + demo + preguntas. **Regla de oro:** en la slide van pocas palabras; lo que explica el detalle es lo que DECÍS.

Reparto por integrante

Integrante	Bloque	Slides	Detalle en la guía (PDF)
(quien abra)	Apertura + arquitectura	1–5	—
Tayra	Eureka + Gateway + Docker	6–8	pág. 3 · 11–12 · 26
Jason	Seguridad / JWT + users	9–11	pág. 4–5 · 6–10
Curi	Catálogo + Reseñas	12–13	pág. 13–15 · 23–24
Cristofer	Checkout distribuido (Feign + saga) + Cupón	14–16	pág. 16–18 · 27–28
Sergio	RabbitMQ + Pago + Promos	17–19	pág. 19–20 · 21–22 · 25
(cierre, todos)	Frontend + demo + conclusiones	20–24	—

Bloque 0 — Apertura

Slide 1 — Portada

En la slide: Logo Krypton · "Migración de un Monolito a Microservicios" · CIBERTEC — DAW II / EFSRT VI · Integrantes (Tayra, Jason, Curi, Cristofer, Sergio) · Fecha.  **Qué decir:** Presentarse, una frase de qué van a mostrar: "cómo tomamos un e-commerce monolítico y lo convertimos en 9 microservicios".  **Visual:** Portada limpia con el logo. Paleta de la marca.

Slide 2 — Agenda

En la slide: 1) Qué es Krypton · 2) Por qué microservicios · 3) La arquitectura · 4) Cada servicio y patrón · 5) Demo · 6) Conclusiones.  **Qué decir:** El recorrido en 30 segundos.  **Visual:** Lista numerada / timeline horizontal.

Slide 3 — El proyecto Krypton

En la slide: E-commerce B2C de tecnología · Monolito Spring Boot 3 + Java 17 · Frontend React 19 · MySQL 8 · Catálogo, carrito, checkout, órdenes, admin.  **Qué decir:** El punto de partida: una app que YA funciona (monolito, 411 tests verdes). No reescribimos la lógica; la repartimos.  **Visual:** Captura de la tienda + el stack en íconos.

Slide 4 — ¿Por qué microservicios?

En la slide: Monolito = todo en un proceso · no se escala por partes · un fallo afecta todo · despliegue acoplado. → Microservicios: cada parte independiente, su propia base, escala y se despliega sola.  **Qué decir:** El problema del

monolito y qué queremos demostrar: **Eureka** (descubrimiento), **Feign** (llamadas síncronas), **RabbitMQ** (eventos), **Docker**. 📺 **Visual:** Izquierda "monolito" (un bloque) vs derecha "microservicios" (varios bloques). Antes/después.

Slide 5 — Arquitectura general

En la slide: El diagrama: Frontend → **api-gateway (:8080)** → 9 servicios, todos registrados en **eureka (:8761)**, cada uno con su base MySQL, + **RabbitMQ**. Las 3 ideas: 1 base por servicio · se llaman por NOMBRE · el JWT es el pasaporte. 🗝 **Qué decir:** La foto completa. A partir de acá, cada uno explica su parte. 📺 **Visual: EL diagrama clave** (el de la guía/README). Que se entienda de un vistazo.

Bloque Tayra — Infraestructura (*Eureka · Gateway · Docker*)

Slide 6 — Service Discovery (Eureka)

En la slide: Problema: las IPs/puertos cambian, no se hardcodean · Solución: un **registro** vivo · cada servicio se anota al arrancar y los demás lo buscan por nombre. 🗝 **Qué decir:** Analogía de la **guía telefónica**: en vez de memorizar el número (IP), buscás por nombre. `eureka-server` es ese registro. 📺 **Visual:** Servicios anotándose en una "agenda" central. Captura del panel de Eureka (`:8761`).

Slide 7 — API Gateway

En la slide: Una sola puerta (`:8080` , donde apunta el front) · rutea por nombre (`lb://users-service`) resolviendo con Eureka · centraliza CORS · el front no conoce los puertos internos. 🗝 **Qué decir:** El portón único. El navegador habla solo con el gateway; él reparte. `lb://` = resolver por Eureka + balancear. 📺 **Visual:** Gateway como puerta con flechas a los servicios.

Slide 8 — Dockerización

En la slide: Un `Dockerfile` por servicio · `docker-compose.full.yml` levanta TODO con un comando · parametrización por env (`localhost` → nombres de contenedor) · 11 contenedores. 🗝 **Qué decir:** Empaquetamos cada servicio en una imagen; un `docker compose up` levanta MySQL + RabbitMQ + los 9 servicios + el gateway. El truco: las URLs configurables (en Docker se hablan por nombre de contenedor). 📺 **Visual:** Una caja "Docker" conteniendo los contenedores. La línea `docker compose -f docker-compose.full.yml up -d --build`.

Bloque Jason — Seguridad / JWT (*users-service*)

Slide 9 — Login: el flujo

En la slide: Front → `POST /api/auth/login` → users verifica password (BCrypt) → **emite el JWT** → el front lo guarda → lo manda en cada request. 🗝 **Qué decir:** Recorrer el flujo (referenciar el doc `flujo-login-jwt`): el usuario se loguea una vez, recibe un token, y ese token lo identifica en todo lo demás. 📺 **Visual:** Diagrama del flujo de login (el del documento de flujo).

Slide 10 — ¿Qué es un JWT?

En la slide: `header.payload.signature` · header = algoritmo (HS384) · payload = email + rol + expiración · signature = firma · **firmado, NO encriptado** (cualquiera lo lee, nadie lo modifica). 🗝 **Qué decir:** Mostrar un token

real decodificado (las 3 partes). El payload se lee, pero sin el secreto no se puede falsificar la firma. 📺 **Visual:** Un token coloreado en 3 partes + las 3 partes decodificadas al lado.

Slide 11 — JWT stateless + secreto compartido

En la slide: `users` FIRMA con el secreto · los demás VERIFICAN con el MISMO secreto · sin sesión, sin DB, sin preguntarle a `users` · el secreto compartido = el "contrato". 🗒 **Qué decir:** Por qué escala: cada servicio valida el token por su cuenta (recalcula la firma). El secreto es la clave; mismo secreto = misma firma = confianza. 📺 **Visual:** `users` → token → varios servicios, todos con la misma "llave" (el secreto).

Bloque Curi — Catálogo + Reseñas

Slide 12 — catalog-service

En la slide: Dueño del catálogo (productos, categorías, stock) · base `krypton_catalog` · **valida un JWT que NO emitió** (stateless) · expone endpoints internos de stock para el checkout. 🗒 **Qué decir:** catalog no tiene la tabla `users`, pero valida el token igual (con el secreto compartido). Y guarda el stock, que después el checkout va a necesitar descontar. 📺 **Visual:** catalog con su base + un candado (valida JWT) + una flecha "stock" hacia order.

Slide 13 — review-service (reseñas)

En la slide: Reseñas de productos (rating 1–5 + comentario) · base `krypton_reviews` · **promedio** con agregación (`AVG`) · valida que el producto exista **vía Feign a catalog** · 1 reseña por usuario por producto. 🗒 **Qué decir:** Un servicio de dominio nuevo. Antes de aceptar una reseña, le pregunta a catalog (Feign) si el producto existe. Reusa todo el patrón (base propia, JWT stateless). 📺 **Visual:** Página de producto con estrellas + la flecha Feign `review`→`catalog`.

Bloque Cristofer — El corazón: Checkout distribuido

Slide 14 — El problema del checkout

En la slide: El checkout debe **descontar stock**... pero el stock vive en **catalog** (otra base, otro proceso) · No hay `@Transactional` que cruce dos servicios · ¿Cómo garantizo "todo o nada"? 🗒 **Qué decir:** Este es el desafío central de los microservicios: una operación que toca dos servicios. En un monolito era una sola transacción; acá no. 📺 **Visual:** order y catalog separados, con una "transacción" tachada cruzándolos.

Slide 15 — Feign + Saga con compensación

En la slide: **Feign** = llamar a otro servicio como un método (síncrono) · order pide a catalog descontar stock · si algo falla DESPUÉS → **compensación**: order le pide a catalog **restaurar** el stock · el "rollback distribuido a mano". 🗒 **Qué decir:** El concepto estrella. Descontar primero; si la orden falla, deshacés el descuento llamando a `restore`. Eso es la saga. Lo probamos con tests (happy path, sin stock, compensación). 📺 **Visual:** Diagrama de la saga: `order` → (decrease) `catalog` → crear orden → X falla → (restore) `catalog`. Flechas de ida y de compensación.

Slide 16 — Cupón de descuento (integración real)

En la slide: `couponCode` en el checkout · order pide el descuento a **promo por Feign** · `total = subtotal - descuento + envío` · columna `discount` (Flyway V2) · el descuento es REAL (lo decide el backend, no el front). 🗒 **Qué decir:** Otra llamada Feign en el checkout: `order`→`promo`. El front muestra un preview, pero la plata la decide el backend. 📺 **Visual:** Checkout con el input de cupón + la flecha Feign `order`→`promo` + el total bajando.

Bloque Sergio — Mensajería + Pago + Promos

Slide 17 — RabbitMQ: mensajería asíncrona

En la slide: **Síncrono (Feign)** = llamada telefónica (esperás) · **Asíncrono (RabbitMQ)** = WhatsApp (mandás y seguís) · order **publica** "se creó una orden" → notification lo **consume** por su cuenta · desacople total. 🛠 **Qué decir:** La otra forma de comunicarse. order avisa al aire y sigue; no sabe ni espera quién escucha. Podés sumar consumidores sin tocar order. (Referenciar el doc `rabbitmq-en-krypton`.) 📊 **Visual:** order → [exchange → cola] RabbitMQ → notification. Teléfono vs WhatsApp como ícono.

Slide 18 — payment-service

En la slide: Pago de una orden (`POST /api/orders/{id}/pay`) · order delega el cobro a **payment por Feign** · **máquina de estados:** PENDIENTE → CONFIRMADA · 402 si se rechaza · 422 si la transición no es válida. 🛠 **Qué decir:** El pago vive en su propio servicio. order lo llama (Feign) y, si se aprueba, transiciona la orden validando con una máquina de estados (no podés pagar dos veces). 📊 **Visual:** Diagrama de estados de la orden (PENDIENTE→CONFIRMADA→ENVIADO→ENTREGADO / CANCELADA).

Slide 19 — promo-service

En la slide: Cupones de descuento · tipo PORCENTAJE o MONTO · admin crea/lista · `POST /api/promos/apply` calcula el descuento · base `krypton_promos`. 🛠 **Qué decir:** El servicio de cupones. Un CRUD admin + el cálculo del descuento que usa el checkout (slide 16). 📊 **Visual:** Tabla de cupones + ejemplo "10% sobre S/2799 → -S/279.90".

Bloque de cierre — todos

Slide 20 — Frontend conectado

En la slide: React 19 + Vite · todo pega al gateway (:8080) · lo NUEVO: reseñas en el producto · cupón en el checkout · pago de la orden · panel admin de cupones. 🛠 **Qué decir:** El front no cambió de puerta (sigue el gateway). Le sumamos las pantallas de los servicios nuevos. 📊 **Visual:** Capturas: página de producto con reseñas + checkout con cupón.

Slide 21 — Calidad y verificación

En la slide: TDD donde aporta (saga, stock, pagos, cupón — tests verdes) · verificación con **curls reales** + chequeo en la DB · todo corre containerizado. 🛠 **Qué decir:** No confiamos en "debería andar": cada fase se verificó con requests reales y revisando la base. Los conceptos críticos (la saga) con tests unitarios. 📊 **Visual:** Captura de los tests en verde + un curl de ejemplo.

Slide 22 — Demo en vivo

En la slide: (sin texto, o un guion de pasos) Login → ver catálogo → dejar una reseña → agregar al carrito → checkout con cupón → pagar → ver la notificación en el log. 🛠 **Qué decir:** Mostrar el flujo completo en vivo. Tener el stack levantado de antes (modo jars o Docker). Plan B: video grabado por si falla la red. 📊 **Visual:** Pantalla en vivo.
Ensayar la demo antes.

Slide 23 — Conclusiones y aprendizajes

En la slide: Patrones logrados: discovery, gateway, JWT stateless, Feign + saga, RabbitMQ, Docker · Trade-offs (stateless = rápido pero sin baja inmediata) · Gotchas resueltos (cookie de RabbitMQ en Docker, el 401 del error-dispatch). 🛠 **Qué decir:** Qué aprendimos: los patrones, pero también que lo distribuido trae problemas nuevos (consistencia, fallos parciales) y cómo los resolvimos. 🇩🇪 **Visual:** Checklist de patrones ✓.

Slide 24 — Cierre + preguntas

En la slide: "¡Gracias!" · repo en GitHub · "¿Preguntas?". 🛠 **Qué decir:** Agradecer, invitar a preguntas, mencionar que el código y las guías están en el repo. 🇩🇪 **Visual:** Logo + QR al repo (opcional).

Tips finales

- **Pocas palabras por slide.** La diapo es el apoyo; el contenido lo decís vos.
- **Ensayen los traspasos** entre integrantes ("...y ahora Jason les cuenta la seguridad").
- **La demo es lo que más impacta** — ensáyenla y tengan plan B (video).
- **El diagrama de arquitectura (slide 5)** es el ancla: vuelvan a él para ubicar cada parte.
- Cada bloque tiene su **detalle en el PDF de la guía** (ver la tabla de reparto arriba) por si alguien pregunta a fondo.